Project Documentation: VeriRISC CPU Controller Unit

Digital Design Lab

Project Overview

This lab implements the **Controller Unit** for the VeriRISC CPU in Verilog HDL [?]. The controller acts as the central state and control decoder [?]. It evaluates the operation code (`opcode`), execution phase (`phase`), and status flags (`zero`) to generate control signals for memory access, program counter updates, instruction register loading, and accumulator management across 8 distinct clock phases [?].

Specifications & Functionality

- **Phase Sequence**: Evaluates 8 discrete phase steps (`3'b000` to `3'b111`) [?].
- **Opcode Width**: 3-bit instruction opcode input (`opcode[2:0]`) [?].
- **Zero Detection Input**: Single-bit status flag (`zero`) indicating accumulator value is zero [?].
- **Control Signals**: Generates 9 distinct control flags governing CPU datapath behavior [?].

Port Summary

Port Name	Direction	Type	Width	Description
`zero`	Input	`wire`	1	Accumulator zero flag signal [?, ?, ?]
`phase`	Input	`wire`	3	Current 8-phase cycle stage [?, ?, ?]
`opcode`	Input	`wire`	3	3-bit instruction opcode [?, ?, ?]
`sel`	Output	`reg`	1	Select instruction address to memory [?, ?, ?]
`rd`	Output	`reg`	1	Enable memory read operation [?, ?, ?]
`ld_ir`	Output	`reg`	1	Load instruction register flag [?, ?, ?]
`halt`	Output	`reg`	1	CPU halt command flag [?, ?, ?]
`inc_pc`	Output	`wire / reg`	1	Increment program counter flag [?, ?, ?]
`ld_ac`	Output	`reg`	1	Load accumulator from data bus [?, ?, ?]
`wr`	Output	`reg`	1	Enable memory write operation [?, ?, ?]
`ld_pc`	Output	`reg`	1	Load program counter flag [?, ?, ?]
`data_e`	Output	`reg`	1	Enable accumulator onto data bus [?, ?, ?]

Table 1: Controller Port Interfaces

—

Verilog Source Code

Design Module (`controller.v`)

```
module controller #(
    parameter HLT = 3'b000,
              SKZ = 3'b001,
              ADD = 3'b010,
              AND = 3'b011,
              XOR = 3'b100,
              LDA = 3'b101,
              STO = 3'b110,
              JMP = 3'b111
)(
    input wire       zero,
    input wire [2:0] phase,
    input wire [2:0] opcode,
    output reg       sel,
    output reg       rd,
    output reg       ld_ir,
    output reg       halt,
    output reg       inc_pc,
    output reg       wr,
    output reg       ld_pc,
    output reg       data_e,
    output reg       ld_ac
);

    reg alu_op;

    always @(*) begin
        // Helper flag for ALU instructions
        alu_op = (opcode == ADD) || (opcode == AND) || (opcode == XOR)
           || (opcode == LDA);

        // Default output values
        sel    = 1'b0;
        rd     = 1'b0;
        ld_ir  = 1'b0;
        halt   = 1'b0;
        inc_pc = 1'b0;
        ld_ac  = 1'b0;
        wr     = 1'b0;
        ld_pc  = 1'b0;
        data_e = 1'b0;

        case(phase)
            3'b000: begin // INST_ADDR
                sel = 1'b1;
            end

            3'b001: begin // INST_FETCH
                sel = 1'b1;
                rd  = 1'b1;
            end

            3'b010: begin // INST_LOAD
                sel   = 1'b1;
                rd    = 1'b1;
                ld_ir = 1'b1;
            end
```

```
            3'b011: begin // IDLE
                sel    = 1'b1;
                rd     = 1'b1;
                ld_ir  = 1'b1;
            end

            3'b100: begin // OP_ADDR
                halt   = (opcode == HLT);
                inc_pc = 1'b1;
            end

            3'b101: begin // OP_FETCH
                rd = alu_op;
            end

            3'b110: begin // ALU_OP
                rd     = alu_op;
                inc_pc = (opcode == SKZ) && zero;
                ld_pc  = (opcode == JMP);
                data_e = (opcode == STO);
            end

            3'b111: begin // STORE
                rd     = alu_op;
                ld_ac  = alu_op;
                ld_pc  = (opcode == JMP);
                wr     = (opcode == STO);
                data_e = (opcode == STO);
            end

            default: begin
                sel    = 1'b0;
                rd     = 1'b0;
                ld_ir  = 1'b0;
                halt   = 1'b0;
                inc_pc = 1'b0;
                ld_ac  = 1'b0;
                wr     = 1'b0;
                ld_pc  = 1'b0;
                data_e = 1'b0;
            end
        endcase
    end

endmodule
```

—

Testbench Module (`controller_test.v`)

```
`timescale 1ns / 1ps

module controller_test;

  localparam integer HLT=0, SKZ=1, ADD=2, AND=3, XOR=4, LDA=5, STO=6,
     JMP=7;
```

```
reg  [2:0] opcode;
reg  [2:0] phase;
reg        zero;
wire       sel;
wire       rd;
wire       ld_ir;
wire       inc_pc;
wire       halt;
wire       ld_pc;
wire       data_e;
wire       ld_ac;
wire       wr;

controller controller_inst (
  .opcode ( opcode ),
  .phase  ( phase  ),
  .zero   ( zero   ),
  .sel    ( sel    ),
  .rd     ( rd     ),
  .ld_ir  ( ld_ir  ),
  .inc_pc ( inc_pc ),
  .halt   ( halt   ),
  .ld_pc  ( ld_pc  ),
  .data_e ( data_e ),
  .ld_ac  ( ld_ac  ),
  .wr     ( wr     )
);

task expect;
  input [8:0] exp_out;
  if ({sel,rd,ld_ir,inc_pc,halt,ld_pc,data_e,ld_ac,wr} !== exp_out)
     begin
    $display("\nTEST FAILED");
    $display("time\topcode phase zero sel rd ld_ir inc_pc halt ld_pc
       data_e ld_ac wr");
    $display("%0d\t%d      %d     %b    %b   %b  %b     %b      %b
          %b     %b      %b     %b",
             $time, opcode, phase, zero, sel, rd, ld_ir, inc_pc, halt
                , ld_pc,
             data_e, ld_ac, wr);
    $finish;
  end
endtask

initial begin
  zero=0;

  $write("Testing opcode HLT phase"); opcode=HLT;
  phase=0; #1 expect (9'b100000000);
  phase=1; #1 expect (9'b110000000);
  phase=2; #1 expect (9'b111000000);
  phase=3; #1 expect (9'b111000000);
  phase=4; #1 expect (9'b000110000);
  phase=5; #1 expect (9'b000000000);
  phase=6; #1 expect (9'b000000000);
  phase=7; #1 expect (9'b000000000);

  $write("\nTesting opcode ADD phase"); opcode=ADD;
```

```
    phase=0; #1 expect (9'b100000000);
    phase=1; #1 expect (9'b110000000);
    phase=2; #1 expect (9'b111000000);
    phase=3; #1 expect (9'b111000000);
    phase=4; #1 expect (9'b000100000);
    phase=5; #1 expect (9'b010000000);
    phase=6; #1 expect (9'b010000000);
    phase=7; #1 expect (9'b010000010);

    $display("\nTEST PASSED");
    $finish;
  end

endmodule
```

—

Simulation & Verification

Execution Command

To run batch simulation using Cadence Xcelium or VCS [**?**]:

```
xrun controller.v controller_test.v
```

Expected Simulation Output

Upon running the testbench, all instructions traverse their 8 phases and trigger the expected control vectors [**?**]:

```
Testing opcode HLT phase 0 1 2 3 4 5 6 7
Testing opcode SKZ phase 0 1 2 3 4 5 6 7
Testing opcode ADD phase 0 1 2 3 4 5 6 7
Testing opcode AND phase 0 1 2 3 4 5 6 7
Testing opcode XOR phase 0 1 2 3 4 5 6 7
Testing opcode LDA phase 0 1 2 3 4 5 6 7
Testing opcode STO phase 0 1 2 3 4 5 6 7
Testing opcode JMP phase 0 1 2 3 4 5 6 7

TEST PASSED
```